2026 臺北秋季程式設計節 • 城市通微服務大黑客松

城市通
入門指引

本次工作坊說明三件事：
城市通開發環境的建置、微服務的開發方式，
以及競賽當天的安排。

賽前工作坊 • 10/25

2026 版



今天的流程

從開源精神到部署上線，共八個單元

01
開源精神
開源理念與 AGPL 授權

02
建置開發環境
版本、工具與建置指令

03
App 與微服務溝通
雙向溝通與自訂 Handler

04
開發自己的服務
服務設定、設計規範、無障礙

05
從民眾需求出發
案例、開放資料、往屆作品

06
AI 使用指引
開發建議與執行規範

07
部署上線
Firebase、Cloud Run

08
競賽當天
發表、繳交、準備清單

第 04 章最後附有賽前練習，建議於比賽前完成



01

開源精神

城市通將程式碼、設計與文件全部公開，
希望平台與社群一同成長，讓每一屆的作品持續豐富市民服務。

01 · 開源精神

從政府主導，走向市民共創

過去

政府主導

政府規劃、發包、上線，
市民是使用者

現在

全部開源

程式碼、設計與文件公開，
任何人都能 fork 修改

未來

市民提案

市民提出並開發服務，
政府協助優質服務上架

意義 本屆得獎作品將整併進城市通，正是邁向「市民提案」的開始。

開源帶來的四個價值：協作與創新・增強透明度・提升穩定加速迭代・數位自主權

01 · 開源精神

開源帶來的四個價值，也是今年規則的由來

01 協作與創新

任何人都能取得程式碼，依需求修改、擴充，
因此可以直接在城市通上開發新服務。

02 增強透明度

程式碼公開，任何人都能檢視，市民更信任數位服務。
因此參賽作品也會開源。

03 提升穩定、加速迭代

各領域的人一起找漏洞、修 bug，
搭配版本控管、測試與組態管理，更新快又安全。

04 數位自主權

技術和資料由政府和市民共同掌握，自主決定發展方向。
因此請選用開源、免費的模組，AI 推論放在裝置端。

01 · 開源精神

認識 AGPL 授權

AGPL-3.0 重點

- 任何人都能用、改、散布你的程式碼
- 改過的版本，也要用同樣授權把原始碼放出來
- 透過網路提供服務時，也要提供原始碼， 這是它和 GPL 的主要差別

對參賽作品的影響

- 比賽期間的作品放在公開 repo， 並附上主辦指定的授權聲明
- 得獎公布後，repo 請保持公開至少一年
- 得獎隊伍請協助主辦， 將作品整併進城市通

建議

選用套件前請先確認 LICENSE。MIT、BSD、Apache-2.0 皆可納入 AGPL 專案；
標示「僅供個人使用」「不得再散布」者，建議改用其他套件。城市通本身採 GPL-3.0。

來源：競賽須知（115.08.27 版）；TownPass LICENSE（GNU GPL v3）

01 · 開源精神

行為守則：互相尊重、彼此友善

「我們作為社群的成員、貢獻者和領導者，
承諾致力於營造一個安全、尊重且友善的社群環境，
讓所有人都能參與其中。」

不論年齡、體型、身心障礙、種族、性別認同與表現、經驗水平、教育背景、社經地位、國籍、外貌、膚色、宗教信仰或性取向。

提醒

這份守則在競賽現場同樣適用。無論隊友、評審、工作人員或其他隊伍，都請互相尊重；
長時間開發難免疲累，也請彼此體諒。

出自 TownPass 的 CODE_OF_CONDUCT.md，改編自 Contributor Covenant 2.1



02
建置開發環境

本章說明版本資訊、開發工具與建置指令。
目標是在比賽前，於自己的手機上完成一次城市通的建置。

02 • 建置開發環境

整體架構與參賽作品的位置

App

城市通 App

以 Flutter 開發，
已提供帳號、定位、
通知、掃碼

雙向溝通

你的作品

微服務網頁

在 WebView 中執行的
網頁，
語言、框架不限

API

選配

後端

需要存資料或處理邏輯時
再加入，語言不限

抓資料

資料

開放資料

data.taipei、
data.gov.tw、TDX

建議

參賽作品通常是一個微服務網頁，並視需求搭配後端。
App 本身不一定需要修改，但建議熟悉修改與建置的流程。

去年的「底層技術 → 系統 → 前端框架 → Flutter 呈現」四張圖，濃縮成這一張

02 · 建置開發環境

2026 年版本資訊

項目	現在（2026/10）	注意
Flutter	3.47.6（去年 3.24.0）	.fvmrc 已指定版本，用 FVM 安裝最方便
Dart SDK	3.13 以上	pubspec.yaml 的 environment
iOS	最低 iOS 15	UIScene lifecycle、Swift Package Manager
Android	AGP 9.1.0・Kotlin 2.4.0・Gradle 9.3.1	電腦需要 Java 17
微服務前端	語言、框架不限	官方範例採用 Vue 3；React、純 HTML 皆可

建議 TownPass main 已於 10/6 升級至 3.47.6，建議直接 clone 最新版本，較從去年的 fork 升級更有效率。

來源：github.com/taipei-doit/TownPass（.fvmrc、pubspec.yaml、2026/10/6 升級紀錄）；Town Pass 開發文件

02 • 建置開發環境

開發工具與建議規格

VS Code
安裝 Flutter 與前端擴充套件，
AI 編輯器亦可

Android Studio
Android SDK 與模擬器

Xcode
iOS 建置用，需要 Mac

FVM
依 .fvmrc 管理 Flutter 版本

Git
第一天建立 repo，最後設為公開

Docker
將後端封裝為容器，便於移轉

Postman
測試後端 API

Chrome DevTools
以裝置模擬檢視手機版面

建議規格

項目	建議
CPU 核心數	8
記憶體	16 GB
可用硬碟空間	60 GB

建置 iOS 需使用 Mac。
Flutter 3.47 在 Intel Mac 上會顯示
「即將停止支援」的提示，建議使用 Apple Silicon。
若要執行裝置端 AI，請準備實機。

工具清單對照競賽須知的建議技能（Git、Docker、Postman）與 Town Pass 開發文件

02 • 建置開發環境

環境建置指令

```
# 1. 取得原始碼
git clone https://github.com/taipei-doit/TownPass.git
cd TownPass

# 2. 照 .fvmrc 裝 Flutter 3.47.6
fvm install

# 3. 裝套件、產生程式碼
fvm flutter pub get
fvm dart run build_runner build --delete-conflicting-outputs

# 4. 開啟模擬器或連接手機後執行
fvm flutter run
```

build_runner 的用途

產生 assets.gen.dart（圖示、Mock 資料的路徑）和 JSON 序列化的程式碼。新增圖示後需重新執行。

`--delete-conflicting-outputs`

清除與舊產出衝突的檔案，避免建置錯誤。

提醒 首次建置需要較長時間，iOS 會先下載 Swift Package，請耐心等候。

未使用 FVM 時，移除每行開頭的 fvm 即可，並確認 flutter --version 為 3.47.6

02 • 建置開發環境

專案結構與常用修改位置

```
TownPass/
├─ lib/
│  ├─ bean/          資料模型
│  ├─ page/          各頁面：city_service、home…
│  ├─ service/       帳號、定位、通知
│  ├─ util/          TP 元件、web_message_handler
│  └─ main.dart
├─ assets/
│  ├─ mock_data/     5 個假資料 JSON
│  ├─ svg/           108 個圖示
│  └─ image/
├─ fonts/            PingFang TC
├─ android/  ios/
└─ .fvmrc            Flutter 3.47.6
```

常用修改位置

新增服務

`lib/page/city_service/model/my_service_model.dart`

擴充 App 能力

`lib/util/web_message_handler/`

切換測試帳號

`assets/mock_data/account.json`

依 taipei-doit/TownPass main 分支（2026/10）整理

02 • 建置開發環境

給去年 fork 過的隊伍：五項主要更新

更新了什麼	建議怎麼做
Flutter 3.32.8 → 3.47.6（2026/10/6）	從最新 main 重新 fork，再把自己的改動搬過去
iOS 最低版本拉到 15，改用 UIScene lifecycle	改過 AppDelegate 或用到舊外掛的，需要手動遷移
Android 工具鏈升級：AGP 9.1.0、Kotlin 2.4.0、Gradle 9.3.1	裝好 Java 17；build 有狀況先試 flutter clean
iOS 改用 Swift Package Manager	CocoaPods 已進入維護模式，外掛優先選支援 SPM 的
Material / Cupertino 拆成獨立套件（material_ui、cupertino_ui）	內建版 11 月才會標示棄用，比賽期間可以先照舊

建議 建議從最新版本重新 fork，再將自行修改的檔案移入，較直接升級更有效率。

來源：TownPass 2026/10/6 升級 commit；What's new in Flutter 3.47（2026/8/12）

02 · 建置開發環境

使用 Mock 資料進行測試

assets/mock_data/account.json（節錄）

```
{
  "name": "TP_SUCCESS",
  "status": 0,
  "data": {
    "username": "Wesley",
    "realName": "金大森",
    "verifyLevel": "3",
    "addresses": [{ "city": "臺北市", … }],
    "citizen": true,
    "nativePeople": false
  }
}
```

account.json	帳號資料，微服務呼叫 userinfo 時回傳的內容
activity.json	首頁活動
home_banner.json	首頁橫幅
message.json	訊息頁
subscription.json	訂閱範例

建議 若要展示「非市民」或「長者」情境，修改 citizen、birthday 即可切換身分。

AccountService 用 rootBundle 讀 account.json；裡面的個資都是測試用的假資料

WebView 開發注意事項

釋放記憶體

切換頁面或 App 進入背景時，
請釋放 WebView 資源，
以維持運作順暢。

雙平台測試

彈出視窗在 iOS 預設會被阻擋，
Android 需自行處理，
建議兩個平台都完整測試。

於 App 內實測

開發時可用裝置模擬檢視，
完成後請於 App 內實測；
雙向溝通也需在 App 內運作。



03

App 與微服務溝通

微服務網頁在 WebView 中執行，
可透過 flutterObject 請 App 執行定位、掃碼、發送通知，
並將結果傳回網頁。

03 · App 與微服務溝通

雙向溝通的運作流程

微服務網頁（WebView）

① postMessage()
把 { name, data } 轉為 JSON 字串送出

物件名：**flutterObject**
格式：**{ name, data }**，以字串傳送
原始碼：**lib/util/web_message_handler/**

⑤ onmessage
從 event.data 取得 JSON 字串並解析

城市通 App（Flutter）

② TPWebMessageListener
解析 JSON，依 name 找到對應的 Handler

③ TPWebMessageHandler
執行定位、掃碼、發通知

④ replyWebMessage()
將結果包裝為 { name, data } 字串回傳

底層是 flutter_inappwebview 的 WebMessageListener

03 · App 與微服務溝通

網頁端的實作方式

```
// 送出：postMessage 傳的是字串
window.flutterObject.postMessage(
  JSON.stringify({ name: 'location', data: null })
);

// 接收：App 回的也是 JSON 字串
const onReply = (event) => {
  const { name, data } = JSON.parse(event.data);
  if (name === 'location') {
    console.log(data.latitude, data.longitude);
  }
};
window.flutterObject.addEventListener('message', onReply);
// 用完記得移除，避免重複觸發
window.flutterObject.removeEventListener('message', onReply);
```

提醒

一般瀏覽器中沒有 flutterObject，
建議先判斷是否存在，
必要時改用模擬資料。

官方範例

官方 Vue 微服務已封裝為 composables：

`src/composables/`
`useConnectionMessage.ts`
`useHandleConnectionData.ts`

來源：Town Pass 開發文件「Two-Way Connection on Website」

03 · App 與微服務溝通

以自訂 Handler 擴充 App 能力

```dart
class GreetingMessageHandler extends TPWebMessageHandler {
  @override
  String get name ⇒ 'greeting';

  @override
  Future<void> handle({
    required Object? message,
    required WebUri? sourceOrigin,
    required bool isMainFrame,
    required Function(WebMessage reply)? onReply,
  }) async {
    onReply?.call(replyWebMessage(data: '哈囉，$message！'));
  }
}
```

1 **繼承 TPWebMessageHandler**
寫在 tp_web_message_handler.dart 或新增檔案

2 **name 對應網頁送出的 name**
請避免與內建的 location、notify 等名稱重複

3 **加入清單**
加入 TPWebMessageListener 的 _cachedMessageHandlers

4 **網頁端呼叫**
postMessage 傳 name: 'greeting'、data: '市民'

去年叫 CPWebMessageHandler，今年改名 TPWebMessageHandler；路徑 lib/util/web_message_handler/

03 • App 與微服務溝通

內建能力與自行擴充

內建 以 postMessage 直接呼叫

- ✓ 定位
- ✓ 手機通知
- ✓ 撥電話、1999
- ✓ QR Code 掃描
- ✓ 裝置資訊
- ✓ 開地圖、開連結

網頁端傳送對應的 name 即可，原生功能由 App 處理

自己加 以自訂 Handler 擴充

NFC	藍牙	陀螺儀
健康資料	音訊	各種 IoT
Wi-Fi	AR／VR	**裝置端 AI**

延續去年的項目，今年新增裝置端 AI（第 06 章）

擴充流程

1. **選擇開源的 Flutter 套件**
   於 pub.dev 搜尋，
   授權為 MIT／BSD／Apache
2. **封裝為 MessageHandler**
   參考前頁範例，
   自訂 name 並加入清單
3. **設定雙平台權限**
   Info.plist 填寫用途說明，
   AndroidManifest 加入權限

套件請選開源、免費的，符合今年的規則



04

開發自己的服務

本章說明「服務」頁的結構、服務設定的檔案與圖示，
Figma 上的設計規範與無障礙設計，最後附上賽前練習。

04 • 開發自己的服務

「服務」頁的四個區塊



2

1

3

4

1 **我的服務**

使用者自行置頂的常用服務

2 **更多／新增**

進入服務列表選擇置頂服務，請先將你的服務加入列表

3 **服務拼貼**

中央區塊，呈現重要的市政服務

4 **熱門**

使用頻率較高的服務

頁面原始碼：lib/page/city_service/；編輯置頂：lib/page/city_service_edit/

04 · 開發自己的服務

服務清單的設定：my_service_model.dart

```dart
enum MyServiceItemId {
  // …現有的 21 個服務
  zoo,
  myService,  // ① 新增 id
  ;
}

// ② 在 MyServiceIdExt 的 switch 補一個 case
MyServiceItemId.myService ⇒ MyServiceItem(
  title: '我的新服務',
  description: '一句話介紹這個服務',
  icon: Assets.svg.iconMyService.svg(),
  category: MyServiceCategory.cityLife,
  destinationUrl: 'https://my-service.web.app/',
),
```

MyServiceItem 的欄位

`title`	服務名稱
`description`	一句話說明
`icon`	SVG 圖示
`category`	服務列表中的分類
`destinationUrl`	點選後開啟的微服務網址
`forceWebViewTitle`	選填，WebView 的標題

五個分類

市政服務　防疫醫療　城市生活　探索臺北　其他服務

檔案：lib/page/city_service/model/my_service_model.dart

04 • 開發自己的服務

將服務加入城市通

1 **新增圖示**

將 SVG 放入 assets/svg/，檔名使用 snake_case，並執行 build_runner

2 **新增 id**

在 MyServiceItemId enum 新增一行

3 **填寫內容**

於 MyServiceIdExt 新增 case，destinationUrl 填入服務網址

4 **執行並加入**

執行 flutter run，於「服務 › 更多」加入我的服務



右圖：完成後，「我的服務」會出現新服務

04 · 開發自己的服務

設定「熱門」與「服務拼貼」

熱門

model/trending_service_model.dart

```
static List<TrendingService> get serviceList {
  return [
    // …找地點、市政資訊儀表板、施工資訊
    TrendingService(
      title: '我的新服務',
      icon: Assets.svg.iconMyService.svg(),
      url: 'https://my-service.web.app/',
    ),
  ];
}
```

服務拼貼

widget/official_service_card/

市政服務 top_left	**有話要說・警政報警** top_right
防疫醫療・市政 APP bottom_left	**城市生活** bottom_right

四張卡片各為一個 Widget，onTap 目前仍為 TODO，
填入網址即可導向你的服務。去年採 JSON 設定，今年改為 Widget。

路徑都在 lib/page/city_service/

04 · 開發自己的服務

圖示的新增與使用

1 放入檔案

`assets/svg/icon_zoo.svg`

→ build_runner

2 自動產生

`lib/gen/assets.gen.dart`

→ 程式中取用

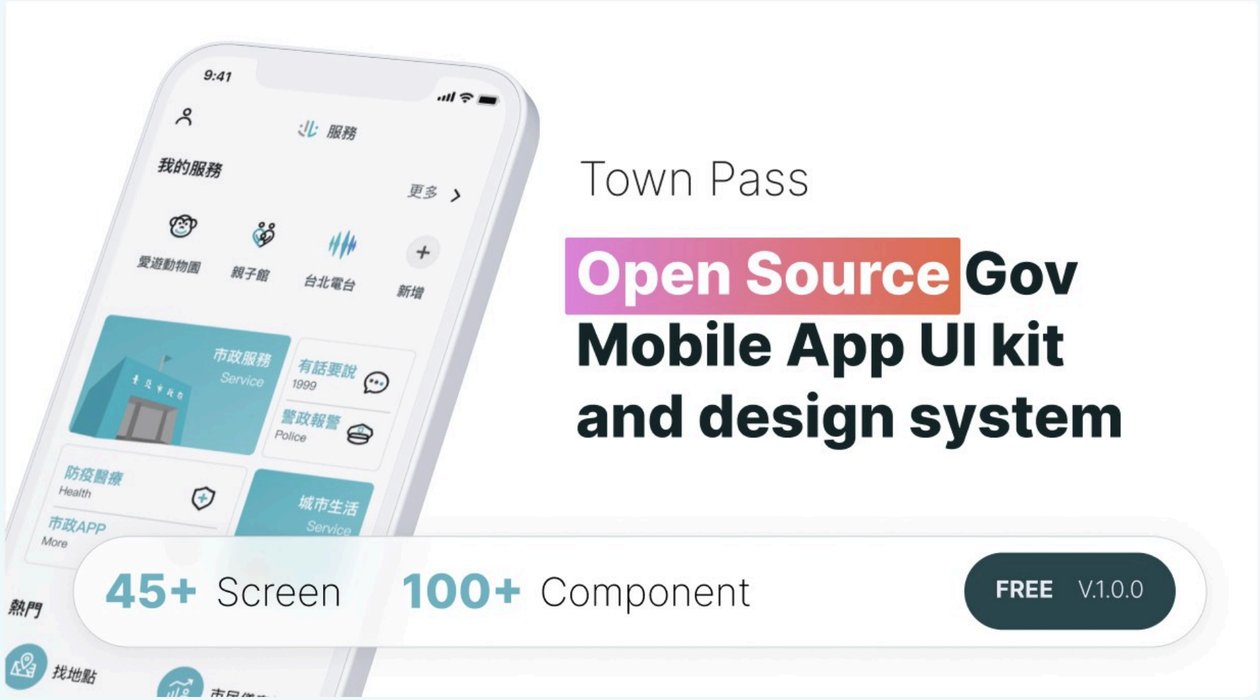
3 取用

```
Assets.svg.iconZoo.svg()
```

- 使用 SVG
- 檔名 snake_case
- 放 assets/svg/
- 新增後重新 build

再一例：icon_dashboard_reports.svg → Assets.svg.iconDashboardReports.svg()；專案中已有 108 個圖示可直接使用

Town Pass 設計資源（Figma）



Town Pass UI Kit

45+ 畫面　　**100+** 元件

免費提供色彩、字級與元件。
設計師可先以 UI Kit 組成畫面，
工程師依此實作，風格與城市通一致。

建議 介面是評審的第一印象，善用 UI Kit，能讓作品與城市通風格一致。

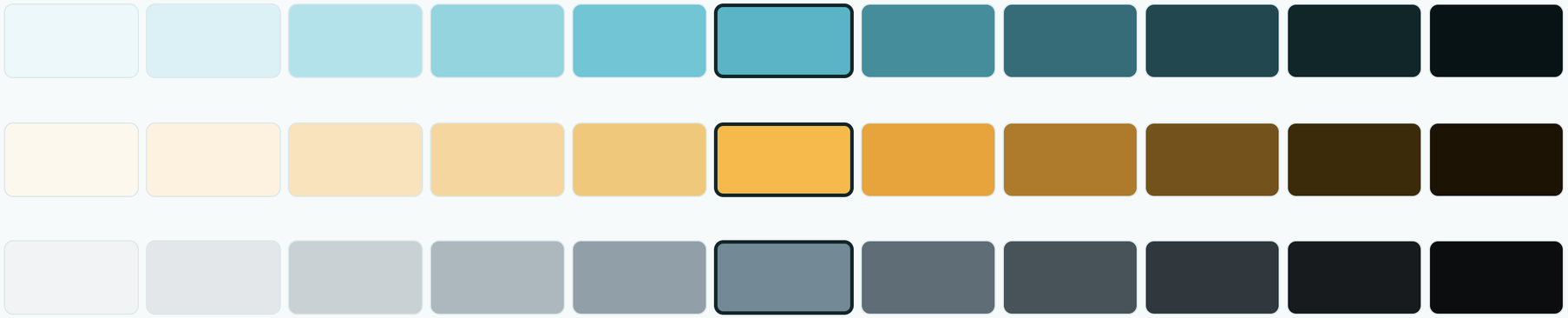
04 · 開發自己的服務

色票與字級規範

Primary 主色
500 #5AB4C5

Secondary 輔色
500 #F5BA4B

Grayscale 灰階
500 #738995



50 → 950，粗框是 500（TPColors.primary500 這些）

字體

中文 PingFang TC，英數 Roboto

字級（TPTextStyles）

h1 36 · h2 24 · title 18 · h3 16 · body 14 · caption 12

來源：lib/util/tp_colors.dart、tp_text_styles.dart；微服務網頁也使用同一組色票

04 · 開發自己的服務

城市通的無障礙設計：12 類規範逐項檢視

AT 替代文字
圖示可朗讀，裝飾圖已隱藏

AV 時序媒體
目前無影音與自動播放

NT 通知
系統通知與 Dialog 皆可被朗讀

TS 觸控
觸控目標至少 48dp × 48dp

FC 焦點
焦點順序依上至下、左至右

FM 表單
欄位標籤清楚，鍵盤對應格式

LK 連結
描述明確，重複連結合併

DC 動態內容
不使用閃爍內容

ST 結構
每頁一個 AppBar 標題

AD 可調適
內容順序合乎邏輯

DT 可辨識
文字放大 200% 仍可閱讀

PD 可預期
導覽順序與圖示保持一致

期待 城市通持續推動無障礙設計，也期待各隊在規劃微服務時，一併考量長者、視障者等不同使用者的需求。

來源：Town Pass 開發文件「無障礙設計」（tpe-guideline.web.app/docs/accessibility/）；規範代碼對照數位發展部無障礙規範

使用者的三種進入情境

1

目標明確的使用者

清楚知道要使用的服務，
例如查詢圖書借閱

引導使用者置頂，下次可直接進入

2

需求尚未明確的使用者

有需要，但還不清楚有哪些服務

透過主動推薦，引發使用意願

3

隨意瀏覽的使用者

沒有特定目標，打開 App 瀏覽

放在熱門，內容宜多元

建議 建議先選定服務的主要對象，Demo 依循該情境進行，呈現會更清楚。

賽前練習 ・ 請於比賽前完成

將一個服務
加入你的城市通

- clone 專案、fvm install、完成建置
- 新增 MyServiceItemId 與 MyServiceItem
- 在 App 中加入「我的服務」
- 新增一張 SVG，重新執行 build_runner
- destinationUrl 先填入 https://www.gov.taipei/
- **以自己的手機完成一次測試**



05

從民眾需求出發

完成技術準備後，接著思考要解決什麼問題。
好的微服務多半源自具體的生活需求。
本章介紹兩個案例、可運用的開放資料，
以及三項已整合進台北通的往屆作品。

05 · 從民眾需求出發

官方微服務參考：11 項服務

申辦服務
線上申辦、查進度

找地點
各區日常服務地點

Q&A
常見問題

優惠券
票券、優惠、票夾

臨櫃叫號
看叫號、預約

市政訊息訂閱
訂閱、收通知

有話要說
陳情

災情通報
回報災情和位置

警政報案
線上報案

圖書館服務
館藏、借閱紀錄、分館地圖

繳費服務
線上繳費、紀錄

官方範例採用 Vue 3
框架可自由選擇，
流程與畫面可作為參考

來源：Town Pass 開發文件 tpe-guideline.web.app（Microservices - User Interface）

05 ・ 從民眾需求出發

案例一：「我的申請到底辦到哪了？」

民眾的聲音

「承辦人常常換，問進度時常聽到『等公文』。」

「媽媽想幫長輩申請看護，卻不清楚有哪些規定。」

「很多福利都是聽里長說才知道，希望資訊更好找。」

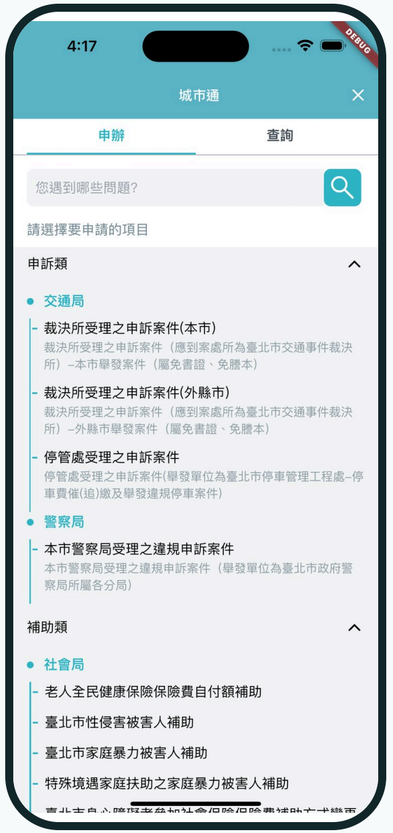
問題分析

申辦資訊分散在各機關、進度不容易掌握，
民眾常常要自己打電話詢問。

切入點

集中各機關的申請項目，一站完成查詢與送件，
並可隨時查詢進度、主動聯繫。

微服務：業務申辦（申請＋案件查詢）



目標：減少民眾往返機關與電話詢問的次數

05 ・ 從民眾需求出發

案例二：「急了，附近到底哪裡有？」



突發狀況

「寶寶哭不停，附近哪裡可以安心哺乳？」

「有人昏倒了！最近的 AED 在哪？」

「手機快沒電，等下要跟客戶通話，
哪裡能借行動電源？」

問題分析

政府提供許多友善據點，
只是資訊分散，急需時要花時間尋找。

切入點

將緊急、必需、友善的據點整合於同一張地圖，
即時查詢、直接聯絡。

微服務：找地點

規則 今年地圖請使用開源方案，例如 Leaflet、MapLibre，搭配 OpenStreetMap 或國土測繪中心圖資。

05 • 從民眾需求出發

兼顧民眾需求與數位治理

民眾需求

日常生活中
對政府服務的具體需求

符合需求

你的微服務

兩者兼顧

公告　優惠　查詢
申請　支付　地圖

數位治理

政府以數位工具
提升效率與透明度

增加效益

六種常見型態可單獨使用，也可組合

05 • 從民眾需求出發

評估作品的六個面向

政府端
數位治理

市民端
民眾需求

安全
個資保護、防攻擊、能被監督

創新
新想法、鼓勵創造、能持續改

有感
民眾有感、解決需求、可擴充

參與
方便表達意見、參與社區、民主自決

便利
數位歸戶、無縫接軌、主動推薦

透明
用語淺白、即時更新、指引清楚

建議 決選問答常圍繞這六個面向，建議每一項都準備簡要的說明。

撰寫 README 或回答評審提問時，可依這六個面向檢視

05 • 從民眾需求出發

城市通的使用者類型

民眾層

核心用戶
經常使用台北通，涵蓋政府服務與社區活動

潛在用戶
有需求，但尚未使用台北通

周邊使用者
只用其中幾個功能

營運層｜資訊局

營運管理團隊
日常營運、技術支援、用戶服務，作品上線後由他們維護

其他機關
想把自己的系統或政策接進來

商家與潛在商家
在平台上經營，或有意加入的商家

建議 建議思考營運團隊接手後的維運方式：資料由誰更新、問題由誰處理。釐清這些，作品將更接近上線。

05 · 從民眾需求出發

開放資料來源

臺北市資料大平臺

`data.taipei`

臺北市各機關的開放資料，
建議優先使用

政府資料開放平臺

`data.gov.tw`

全國各級政府的資料

TDX 運輸資料流通服務

`tdx.transportdata.tw`

公車、捷運、YouBike；
免費註冊取得金鑰，建議提早申請

臺北城市儀表板

`dashboard.gov.taipei`

市政指標視覺化平台，
亦為開源專案（Taipei-City-Dashboard）

規則 資料使用規範

- 核心資料須為合法開放資料，並註明出處
- 非核心的資料可以用模擬資料
- README 與 Demo 標示資料集名稱與連結

規則來源：競賽須知（115.08.27 版）；城市通 App 已內建「市政資訊儀表板」

05 · 往屆作品

往屆作品：已整合進台北通的三項服務

以下三項往屆作品已於賽後整併進台北通，原始碼均公開於 taipei-doit。

2024 停車

ParkFlow 找車位

停哪裡、停多久、會不會被開單

資料

北市好停車即時車位、data.taipei 紅黃線

`townpass2024-parking`

2025 交通

RideFlow 騎妙

5 秒告訴你，現在適不適合騎 YouBike

資料

YouBike 即時、今日施工、
氣象署、環境部 AQI

`townpass2025-RideFlow`

2024 天氣

即時雨晴

我不想淋雨：現在這裡會不會下雨

資料

中央氣象署雷達回波、1 小時雨量預測

`townpass2024-weather`

建議 建議賽前挑選一項 clone 並實際執行，參考往屆隊伍的雙向溝通實作與 README 撰寫方式。

原始碼：github.com/taipei-doit/（repo 名稱如卡片所列）

05 • 往屆作品 • 2024 ParkFlow

ParkFlow 找車位：停車資訊與計時提醒



民眾需求

「附近哪裡還有位子？」

「這條黃線可以停多久？」

「停車場快超時了，我卻忘了。」

主要功能

1. 於地圖顯示停車場剩餘車位與路邊格位狀態
2. 查詢附近 1 公里內的黃線
3. 停車後開始計時，時間將到時由 App 推播提醒

開放資料

北市好停車即時車位、data.taipei 紅黃線

技術

Vue 3、FastAPI；
App 端自訂 timer_set Handler

亮點

即使網頁已關閉，仍可收到提醒。
團隊在 App 端自訂 Handler，以本機通知排程推播，即第 03 章介紹的做法。

github.com/taipei-doit/townpass2024-parking；當年地圖用 Mapbox，今年可以改用 Leaflet 或 MapLibre

05 • 往屆作品 • 2025 RideFlow

RideFlow 騎妙：YouBike 騎乘適合度分析



民眾需求

「騎到了，結果站點沒位子還。」

「這條路今天在施工嗎？」

「這麼熱，現在騎會不會中暑？」

主要功能

1. 將天氣、空品、施工、站點資訊整合為三級建議
2. 預測目的地站 15、30、45 分鐘後的空位數
3. 估算騎乘時間、卡路里與減碳量

開放資料

YouBike 2.0 即時、今日施工、氣象署、環境部 AQI

技術

Vue 3、FastAPI、XGBoost 預測模型

亮點

整合四種資料，最後提供一句明確的建議。
README 完整記載架構圖、環境變數與 AI 使用聲明，便於接手。

github.com/taipei-doit/townpass2025-RideFlow；當年地圖和路線用 Google Maps，今年可以改用開源方案

05 • 往屆作品 • 2024 即時雨晴

即時雨晴：所在地的即時降雨資訊



民眾需求

「預報說會下雨，那我這裡呢？」

「等一下要出門，要不要帶傘？」

「雨什麼時候會停？」

主要功能

1. 以所在地或指定地點為中心，顯示即時雷達回波
2. 樹林、南屯雷達約 1–2 分鐘更新，並可切換全臺
3. 提供未來一小時雨量預測

開放資料

中央氣象署雷達回波圖、1 小時雨量預測

技術

Flask、純 HTML 與 JS、
Leaflet 搭配 OpenStreetMap

亮點

功能聚焦於單一需求，以純 HTML 與 JS 完成，App 端使用內建的 location。
規模精簡，且貼近每位市民的日常。

github.com/taipei-doit/townpass2024-weather；地圖用的是開源方案，今年的規則下一樣適用

05 · 往屆作品

三項作品的共同特點

共同點	找車位	騎妙	即時雨晴
需求明確，一句話說得清楚	停哪裡、停多久	現在適不適合騎	現在會不會下雨
資料皆來自開放資料	北市好停車、紅黃線	YouBike、施工、氣象、AQI	氣象署雷達、雨量
主流程單純，範圍聚焦	找位 → 停車 → 計時	選站 → 分析 → 建議	就一頁
善用 App 原生能力	定位、自加推播	使用者資料	定位
README 完整，便於接手	功能、API、資料來源	架構圖、環境變數、AI 聲明	資料來源、API 設定

⚠ 找車位使用 Mapbox、騎妙使用 Google Maps；依今年規則，地圖請改用 Leaflet 或 MapLibre 等開源方案。

建議 整併時，資訊局將接手參賽作品的程式碼，建議以這五點檢視自己的作品。

整理自三個作品的原始碼和 README



06

AI 使用指引

AI 已是常見的開發工具。本章說明開發階段的使用建議，
以及今年的重要規範：作品執行時，AI 推論須於裝置端完成。

06 • AI 使用指引

開發階段的 AI 應用建議

第一小時

發想題目方向

詢問市民需求與可用的開放資料，
作為發想參考，方向由隊伍決定。

找資料

整理開放資料

解釋欄位、轉換 JSON、
撰寫清洗腳本，快速掌握資料。

寫程式

協助功能開發

Claude Code、Cursor、Copilot 等皆可；
附上 TownPass 文件與範例效果更好。

除錯

協助除錯

建置錯誤訊息完整提供給 AI，
有助於快速找出方向。

介面

產生介面元件

提供色票、字級與 Figma 截圖，
請 AI 產生元件，節省切版時間。

最後兩小時

撰寫文件與整理 repo

資料來源、架構圖、執行方式，
可由 AI 起草，再由隊伍確認。

規則 主辦單位明文允許於開發階段使用雲端 AI 工具，企劃、開發、除錯、整理資料皆可。

來源：競賽須知（115.08.27 版）AI 使用規範；工具名稱僅為舉例，不限定

06 • AI 使用指引

使用 AI 的注意事項

1 **確認資料來源**
AI 有時會產生看似正確的 API 網址、資料集名稱或欄位，請逐一開啟來源確認。

2 **留意套件授權**
詢問地圖相關問題時，AI 常推薦 google_maps_flutter。提問時請先說明需使用開源、免費的套件。

3 **提供最新的版本資訊**
Flutter 3.47、AGP 9 版本較新，AI 的資訊可能較舊。提供 pubspec.yaml 與官方文件，產出會更準確。

4 **隊內先統一工具**
隊伍先約定使用的工具、程式碼風格與 review 分工，後續合併程式碼會更順利。

5 **理解自己的程式碼**
決選設有問答環節。AI 產生的程式碼請至少完整閱讀一次，以便說明設計考量。

建議 建議在 repo 根目錄放置供 AI 參考的說明檔（例如 CLAUDE.md、AGENTS.md），載明今年的規則與專案結構。

06 • AI 使用指引

執行階段規範：AI 推論須於裝置端完成

允許事項

- 開發階段使用雲端 AI： 企劃、撰寫程式、除錯、整理資料
- 執行時的 LLM 推論於裝置端進行
- 一般後端：登入、資料庫、檔案、排行榜、同步
- 預先生成的固定文字、圖片、程式碼， 並如實說明

⚠ 規範事項

- 執行時的推論於裝置端完成， 不使用雲端 API 或遠端模型
- 使用者輸入與作品資料的 LLM 推論， 亦於裝置端完成
- 請勿使用危害國家資通安全清單所列的 AI 工具與模型（含大陸廠牌）

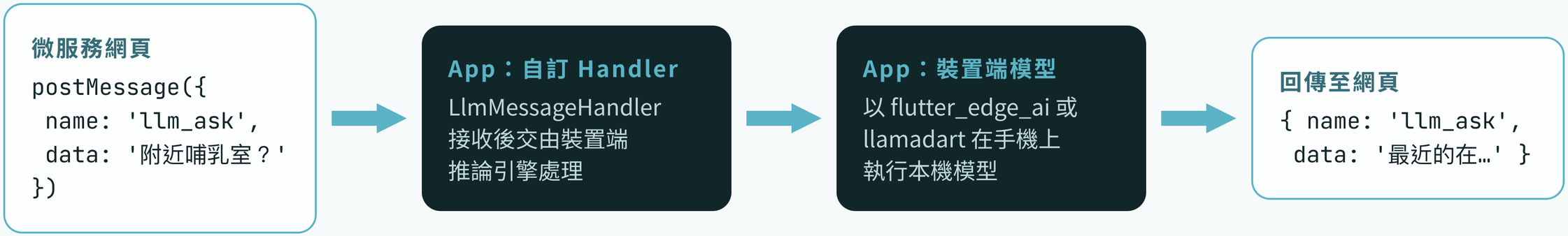
規則

評審可能在**關閉網路**的狀態下現場出題，驗證 LLM 功能；
並請於 README 揭露模型名稱、版本、量化格式、推論框架與來源。

來源：競賽須知（115.08.27 版）AI 使用規範

06 • AI 使用指引

裝置端 AI 的建議架構



整個流程於手機上完成，
離線狀態下也能運作

模型由 App 載入一次，
可供多個微服務共用

將開放資料納入提示詞（RAG），
可提升小模型的回答品質

llm_ask 是示意名稱；做法和第 03 章的自訂 Handler 相同

06 • AI 使用指引

套件與模型的選擇

Flutter 套件	支援模型	特色	授權
flutter_edge_ai（原 flutter_gemma）	Gemma 4 E2B / E4B、Gemma 3n、Phi-4 Mini；.litertlm、.task、.onnx	LiteRT-LM 引擎，Android GPU / NPU 加速，多模態、function calling	MIT
llamadart	GGUF（llama.cpp）和 .litertlm；Dart 3.10+、Flutter 3.38+	一套 API 跨 iOS、Android、桌面；Metal / Vulkan 加速	MIT

建議模型：Gemma 4 E2B

2026/4 以 Apache 2.0 授權釋出，
專為行動裝置設計；
E2B 在 8 GB 記憶體手機上較穩定

實測中文表現

小模型的繁體中文表現差異較大，
建議以實際題目測試；
量化格式也會影響速度與品質

模型檔另行存放

數 GB 的模型檔，
可於 App 首次啟動時下載，
或於 Demo 前預先放入實機

來源：pub.dev（flutter_edge_ai 2.3.2、llamadart 0.8.19）；Gemma 4 於 2026/4/2 發布。套件與模型授權以官方頁面為準

主持人的建議

沒有 AI 功能
也能做出好作品

作品的核心在於解決問題。AI 用在適當之處能加分，未使用也不影響作品價值。

加入 AI 功能前，
先釐清它為使用者解決的問題

裝置端模型首次載入需數十秒，
Demo 前請先預熱

Demo 前以飛航模式完整測試一次



07

部署上線

微服務部署後取得網址，即可由 App 開啟。
前端使用 Firebase Hosting、後端使用 Cloud Run，
各以一個指令完成部署；去年的 gcr.io 流程已更新。

07 · 部署上線

部署方式與平台選擇

微服務網頁（靜態檔）

Firebase Hosting

上傳前端 build 產出的靜態檔，
自動啟用 HTTPS，
並提供 web.app 網址。

亦可使用 GitHub Pages、
Cloudflare Pages、Vercel，
須支援 HTTPS

後端 API（有的話）

Cloud Run

以容器部署、依請求計費，
閒置時不產生費用；
支援 Java、Node、Python 等。

亦可使用 Render、Fly.io，
或於比賽期間以 ngrok
對外提供本機服務

為什麼使用容器

可移轉、可維運、可上架

規則中的這三項，
意指資訊局取得 repo 後，
能順利執行與維運；
準備 Dockerfile 即可達成。

建議 建議賽前先部署一個 Hello World，並將網址填入 destinationUrl 測試一次，比賽當天即可專注於功能開發。

Firebase、GCP 免費額度足夠比賽使用；部署平台不限，請使用免費方案

07 · 部署上線

以 Firebase Hosting 部署前端

```
# 0. 於 console.firebase.google.com 建立專案

# 1. 安裝 CLI 並登入
npm install -g firebase-tools
firebase login

# 2. 在微服務專案根目錄初始化（public 填 build 輸出資料夾）
firebase init hosting

# 3. 建置後部署
npm run build   # 依你用的框架
firebase deploy --only hosting

# 完成：https://你的專案.web.app
```

初始化時的三項設定

public：填入 build 輸出資料夾（Vite 為 **dist**）
single-page app：選 **Yes**
覆蓋 index.html：選 **No**

取得網址後

填入 my_service_model.dart 的 destinationUrl，
執行 flutter run，並從 App 開啟確認。

提醒 部署後請在 App 內測試雙向溝通，此功能需於 App 內才能運作。

Firebase Hosting 免費方案足夠比賽使用；Vite 專案預設輸出目錄為 dist

07 · 部署上線

以 Cloud Run 部署後端

```
# 0. 建立 GCP 專案，安裝 gcloud CLI 並登入
gcloud auth login
gcloud config set project [PROJECT-ID]

# 1. 於後端專案根目錄執行（建議附 Dockerfile）
gcloud run deploy my-service --source . --region asia-east1

# 首次執行會詢問：是否啟用 API → y；是否允許未驗證存取 → y
# 完成：https://my-service-xxxx.asia-east1.run.app
```

--source 的用途

由 Cloud Build 在雲端建置容器：
有 Dockerfile 時依其設定，
未提供時以 buildpacks 自動偵測語言，
再推送至 Artifact Registry。

Container Registry 已停止服務

Container Registry 已於 2025 年 3 月關閉。
去年「docker build -t gcr.io/…」的流程，
請改為推送至 Artifact Registry（pkg.dev）。

區域建議選擇 asia-east1

位於彰化，臺灣地區延遲最低。

來源：Cloud Run「Deploy from source」文件；Container Registry shutdown（2025/3/18）

07 · 部署上線

後端範例：Spring Boot 與 Dockerfile

```
# Dockerfile（置於專案根目錄）
FROM eclipse-temurin:21-jdk AS build
WORKDIR /app
COPY . .
RUN ./mvnw -q package -DskipTests

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
ENV PORT=8080
CMD ["java", "-jar", "app.jar"]
```

1 **於 start.spring.io 建立專案**
Spring Boot 4.x、Java 21、Maven，加入 Spring Web

2 **撰寫 @RestController**
GET /hello 回傳 Hello, World

3 **server.port 讀取環境變數 PORT**
application.properties：server.port=${PORT:8080}。
Cloud Run 會提供 PORT，服務需依此埠號啟動

4 **設定 CORS**
網頁位於 web.app、API 位於 run.app，
屬不同網域，需設定 CORS

Spring Boot 4.1 於 2026 年發布，需 Java 17 以上；使用其他語言的隊伍，改用對應的 Dockerfile 即可

07 • 部署上線

將部署後的服務串回城市通

1 **將 destinationUrl 更新為正式網址**

`destinationUrl: 'https://你的專案.web.app/'`

2 **以 flutter run 安裝至實機**

模擬器亦可，Demo 建議使用實機，定位、掃碼、通知才能完整呈現

3 **服務 › 更多 › 加入我的服務**

開啟服務，確認雙向溝通正常

4 **完整測試一次流程**

開啟 App → 找到服務 → 完成操作 → 回到首頁，這也是 Demo 的流程



建議 Demo 用的手機請事先安裝 App、加入服務並確認網路，上台即可直接展示。



08

競賽當天

最後說明競賽當天的安排：
發表方式、繳交項目、得獎後的整併，以及賽前準備清單。

08 · 競賽當天

發表方式與時間配置

規則 發表流程

- 第二天：初選每隊約 2 分鐘，決選約 5 分鐘＋問答
- 投影作品實際運作的畫面，**不使用簡報**
- 各隊須上台發表，方可參加評選

建議 2 分鐘發表建議

0:00	題目與選定的需求（一句話）
0:15	使用情境：誰、在什麼時候使用這項服務
0:30	實機操作：從城市通進入，完成一條流程
1:40	資料來源與技術實作（20 秒）

主辦建議的發表內容：說明題目 → 創意來源與應用場景 → 開發項目、資料來源與技術細節 → 實際運作展示。
順序可調整，建議四項皆涵蓋。

來源：競賽須知（115.08.27 版）第六、七條

08 · 競賽當天

繳交項目

✓ 所有原始碼

包含微服務、後端與修改後的 TownPass

✓ LICENSE：AGPL-3.0

附上主辦指定的授權聲明，並維持公開一年

✓ README：執行說明

依 README 即可完成建置，便於資訊局接手

✓ 資料來源清單

各資料集的名稱、連結，並註明模擬資料

✓ 第三方套件清單

列出名稱和授權，方便確認都是開源、免費的

✓ 使用 AI 時：模型揭露

模型名稱、版本、量化格式、推論框架、來源

建議 建議第一天即建立 repo 並設為公開。README 可由 AI 起草，資料來源與套件清單請逐條確認。

來源：競賽須知（115.08.27 版）第五、六、十條；後四項為建議項目

08 · 競賽當天

得獎後的整併與經驗分享

2 週內

收到整併通知

主辦單位以 email 說明整併方式

整併

以小幅修改、刪減為主

以比賽期間的作品為範圍；
有任何困難都可以提出

之後

回來分享經驗

得獎隊伍將回到黑客松分享經驗

提醒

得獎隊伍請協助主辦，將作品整併進城市通。
比賽期間依規則開發，整併時會更加順利。

來源：競賽須知（115.08.27 版）

08 · 競賽當天

賽前準備清單（11/7 前）

環境

- ✓ 每位成員完成 TownPass 最新版建置
- ✓ 至少一台實機安裝自行建置的 App
- ✓ Hello World 部署完成，可從 App 開啟
- ✓ 完成 Firebase、GCP、TDX 帳號註冊

隊伍

- ✓ 建立 repo，約定分支與 review 分工
- ✓ 統一 AI 工具與說明檔
- ✓ 預先瀏覽 data.taipei、TDX 的資料集
- ✓ 預先試用地圖、圖表的開源套件

當天攜帶

- ✓ 政府核發之身分證件（報到時查驗）
- ✓ 延長線、手機線、投影轉接頭
- ✓ Demo 用手機兩支（含備用）
- ✓ 個人電腦和設備請自行保管

建議 建議優先完成「環境」欄，比賽開始後即可投入功能開發。

證件驗證、設備自行保管為競賽須知規定；其餘為建議

08 · 競賽當天

相關資源

TownPass 原始碼
github.com/taipei-doit/TownPass

開發文件（中英）
tpe-guideline.web.app

活動官網、競賽須知
codefest.taipei/2026-fall

Figma UI Kit
在開發文件首頁有連結，搜 Town Pass Design Tokens

開放資料
data.taipei · data.gov.tw · tdx.transportdata.tw

城市儀表板（也開源）
github.com/tpe-doit/Taipei-City-Dashboard

Flutter 3.47 更新說明
flutter.dev/blog/whats-new-in-flutter-3-47

問題諮詢
技術問題請開 GitHub issue；規則問題請洽承辦單位（電腦公會）

主辦：臺北市政府資訊局・承辦：台北市電腦商業同業公會

結語

賽前完成環境建置，功能聚焦、做得穩定，11/7 天母見。

2026 臺北秋季程式設計節

城市通微服務大黑客松

主辦：臺北市政府資訊局・承辦：台北市電腦商業同業公會・2026.10 賽前工作坊